Unit III Searching and Sorting

Prof. Pranjal Pandit

Searching and Sorting

Searching:

- sequential/linear search,
- sentinel search,
- binary search,
- Fibonacci search,
- indexed sequential search.

Sorting:

- internal vs external sorting;
- sort order, stability, efficiency and number of passes;
- comparison-based methods (Bubble, Insertion, Selection, Quick, Shell);
- non-comparison-based methods (Radix, Counting, Bucket);
- comparison of all methods and their complexities.

Quicksort

- As the name suggests, Quicksort is one of the fastest sorting algorithms.

- The Quicksort algorithm takes an array of values, **chooses one of the values as the 'pivot' element,** and moves the other values so that **lower values are on the left of the pivot element**, and higher values are on the right of it.

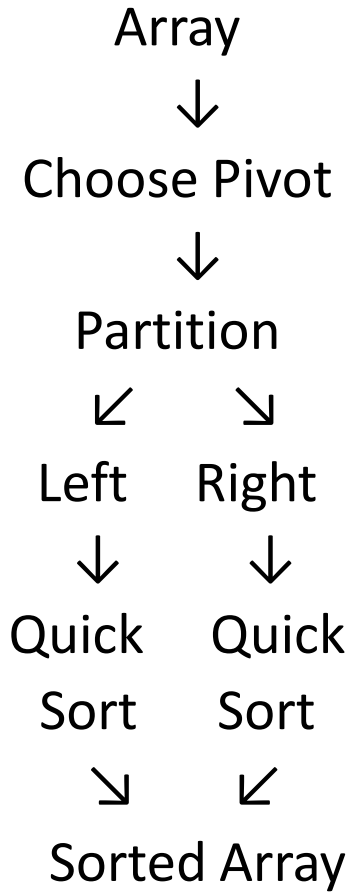
- In **this example the last element** of the array is chosen to be the **pivot element**, but we could also have chosen the first element of the array, or any element in the array really.

- Then, the Quicksort algorithm does the **same operation recursively on the sub-arrays to the left and right side of the pivot element**. This continues until the array is sorted.

How it works:

- Choose a value in the array to be the pivot element.
- Order the rest of the array so that lower values than the pivot element are on the left, and higher values are on the right.
- Swap the pivot element with the first element of the higher values so that the pivot element lands in between the lower and higher values.
- Do the same operations (recursively) for the sub-arrays on the left and right side of the pivot element.

Basic Idea


Array
Choose Pivot
Partition
Left
Right
Quick Sort
Quick Sort
Sorted Array

Example

Consider the array:

10 7 8 9 1 5

Let's use the **last element as the pivot**.

So: Pivot = 5

We now compare every element with 5.

Step 1: Partition around 5

Original: 10 7 8 9 1 5

Elements smaller than 5 should come before 5.

Only 1 is smaller than 5.

After partitioning: 1 5 8 9 10 7

Now 5 is in its **correct final position**.

So we recursively sort:

Left part: 1

Right part: 8 9 10 7

2. Sorting the Right Part

Right part: 8 9 10 7

Choose last element as pivot:

Pivot = 7

All elements are greater than 7, so after partition:

7 9 10 8

Now:

Left: empty

Right: 9 10 8

3. Sort 9 10 8

Choose: Pivot = 8

Both 9 and 10 are greater than 8.

After partition: 8 10 9

Now sort:

10 9

Choose: Pivot = 9

Since 10 > 9:

9 10

Therefore:

1 5 7 8 9 10

Final sorted array 1 5 7 8 9 10

Quick Sort Algorithm

A simple algorithm using the **last element as pivot**:

QUICKSORT(A, low, high)

1. If low < high:
2. pivotIndex = PARTITION(A, low, high)
3. QUICKSORT(A, low, pivotIndex - 1)
4. QUICKSORT(A, pivotIndex + 1, high)

Partition Algorithm

PARTITION(A, low, high)

1. pivot = A[high]
2. i = low - 1
3. For j = low to high - 1:
4. If A[j] < pivot:
5. i = i + 1
6. Swap A[i] and A[j]
7. Swap A[i + 1] and A[high]
8. Return i + 1

```cpp
#include <iostream>
using namespace std;

// Partition function
int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}
```

```cpp
// Quick Sort function
void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
        int pivotIndex = partition(arr, low, high);

        // Sort left part
        quickSort(arr, low, pivotIndex - 1);

        // Sort right part
        quickSort(arr, pivotIndex + 1, high);
    }
}
```

```
int main()
{
    int arr[] = {10, 7, 8, 9, 1, 5};

    int n = sizeof(arr) / sizeof(arr[0]);

    cout << "Original array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    quickSort(arr, 0, n - 1);

    cout << "\nSorted array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

How the Code Works

The important part is: int pivot = arr[high];

This selects the **last element as the pivot**.

For: 10 7 8 9 1 5

we get: pivot = 5

Then:

```
if (arr[j] < pivot)
```

checks whether the current element is smaller than the pivot.

If yes:

```
i++;
swap(arr[i], arr[j]);
```

moves that element to the left side.

Finally:

```
swap(arr[i + 1], arr[high]);
```

places the pivot in its correct position.

Then recursion:

```
quickSort(arr, low, pivotIndex - 1);
quickSort(arr, pivotIndex + 1, high);
```

sorts the two remaining portions.

Complexity of Quick Sort

When does worst case occur?

If the pivot is consistently the **smallest or largest element**, the partition becomes highly unbalanced.

For example:

```
1 2 3 4 5 6 7
```

If we always select the last element as pivot:

```
Pivot = 7
```

then almost the entire array remains on one side. This can result in:

```
O(n²)
```

Case	Time Complexity
Best Case	**O(n log n)**
Average Case	**O(n log n)**
Worst Case	**O(n²)**
Space	**O(log n)** average due to recursion

Shell Sort

Example:

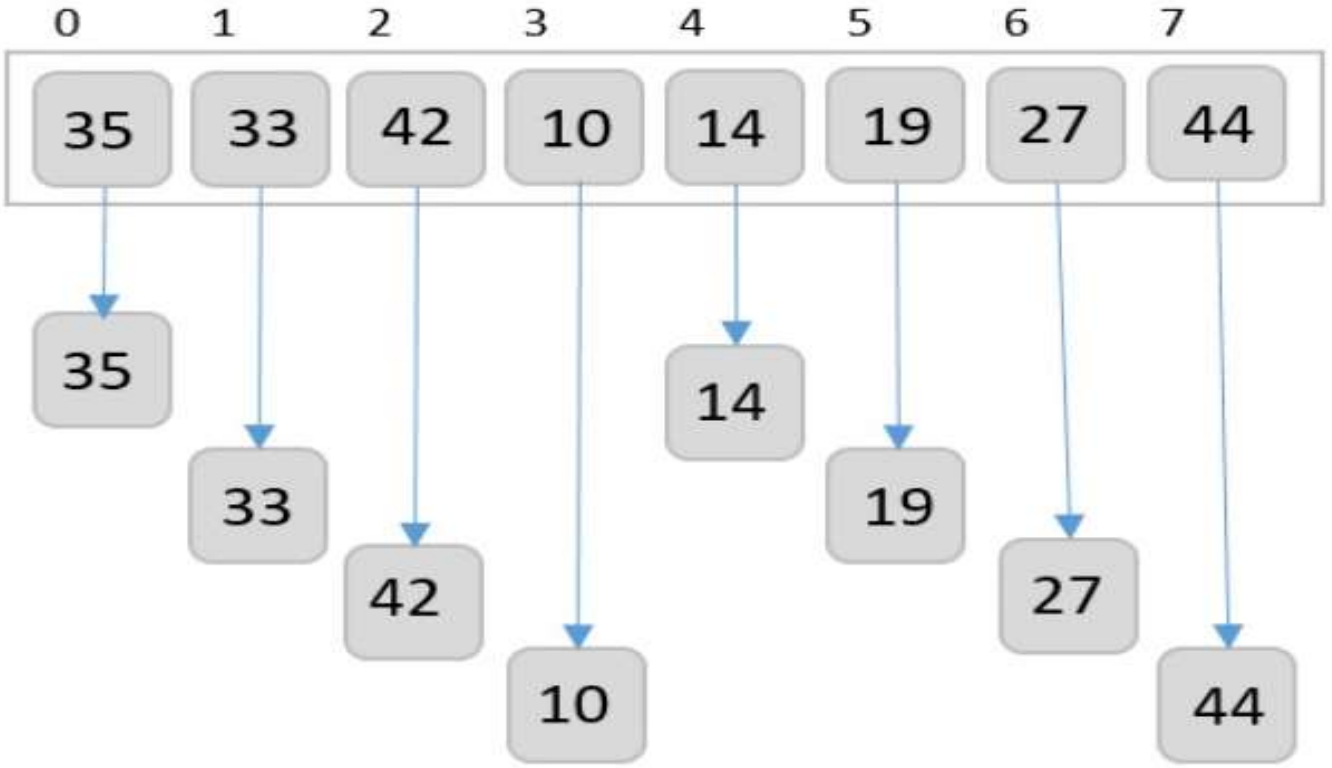
35 33 42 10 14 19 27 44

For our example and ease of understanding, **we take the interval of 4.**

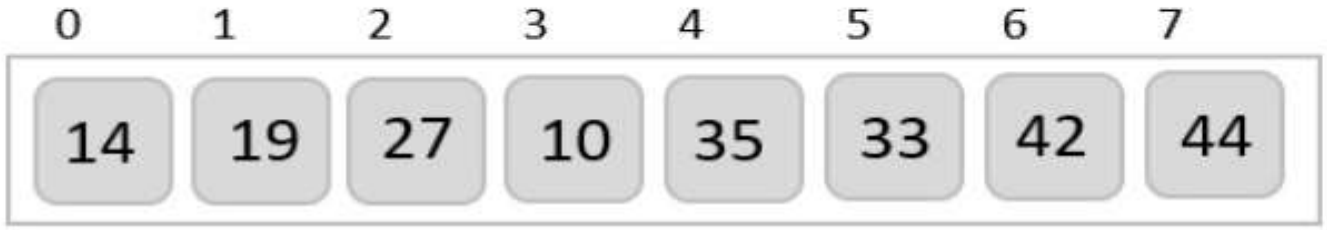
Make a virtual sub-list of all values located at the interval of 4 positions.

Here these values are {35, 14}, {33, 19}, {42, 27} and {10, 14}

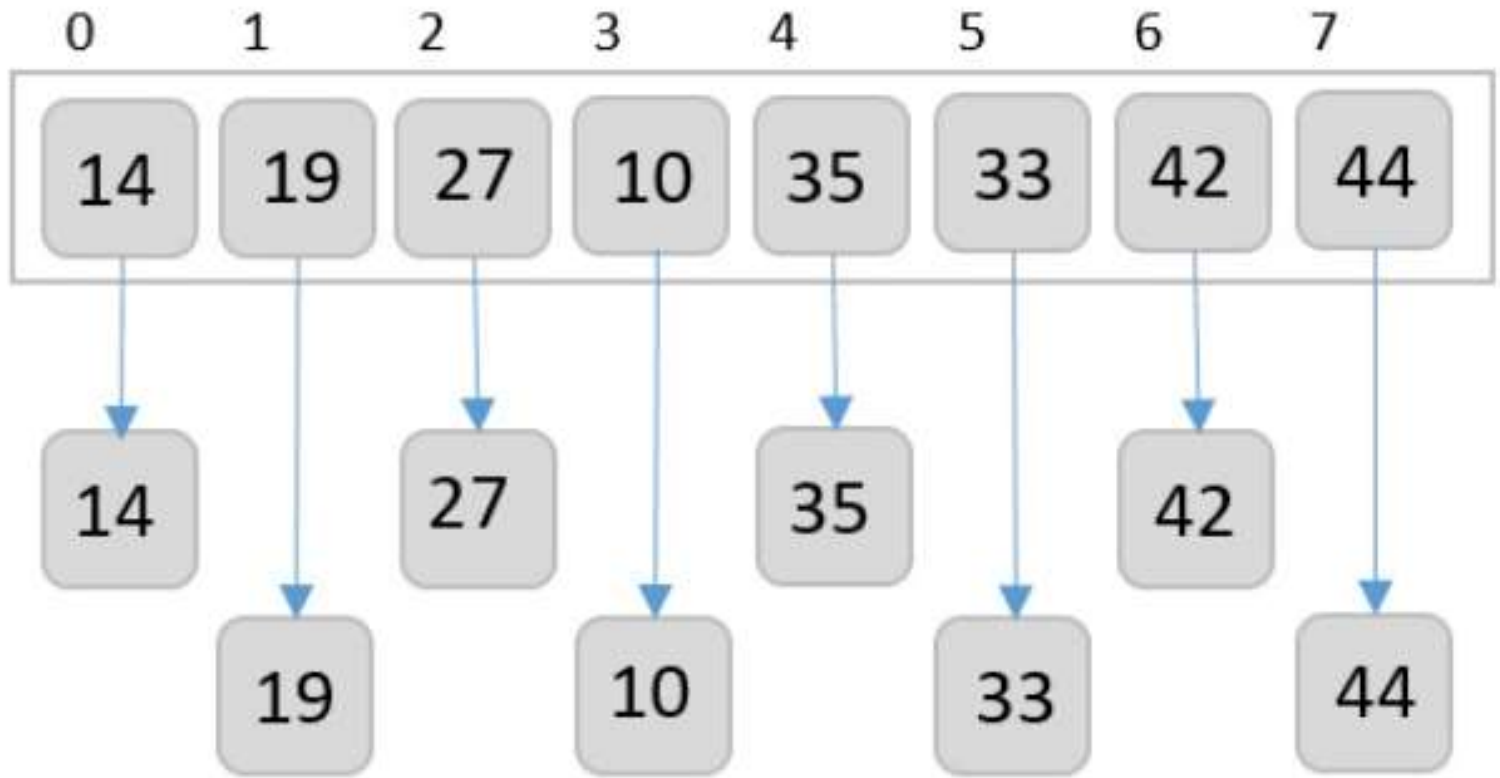
Here these values are {35, 14}, {33, 19}, {42, 27} and {10, 14}



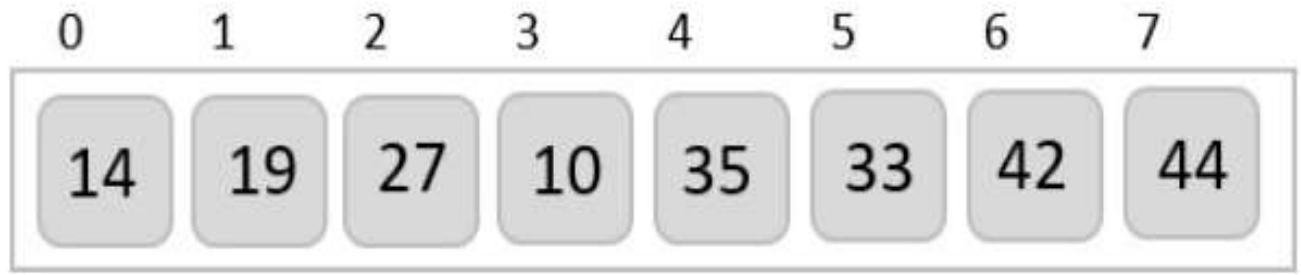
We compare values in each sub-list and swap them (if necessary) in the original array. After this step, the new array should look like this –



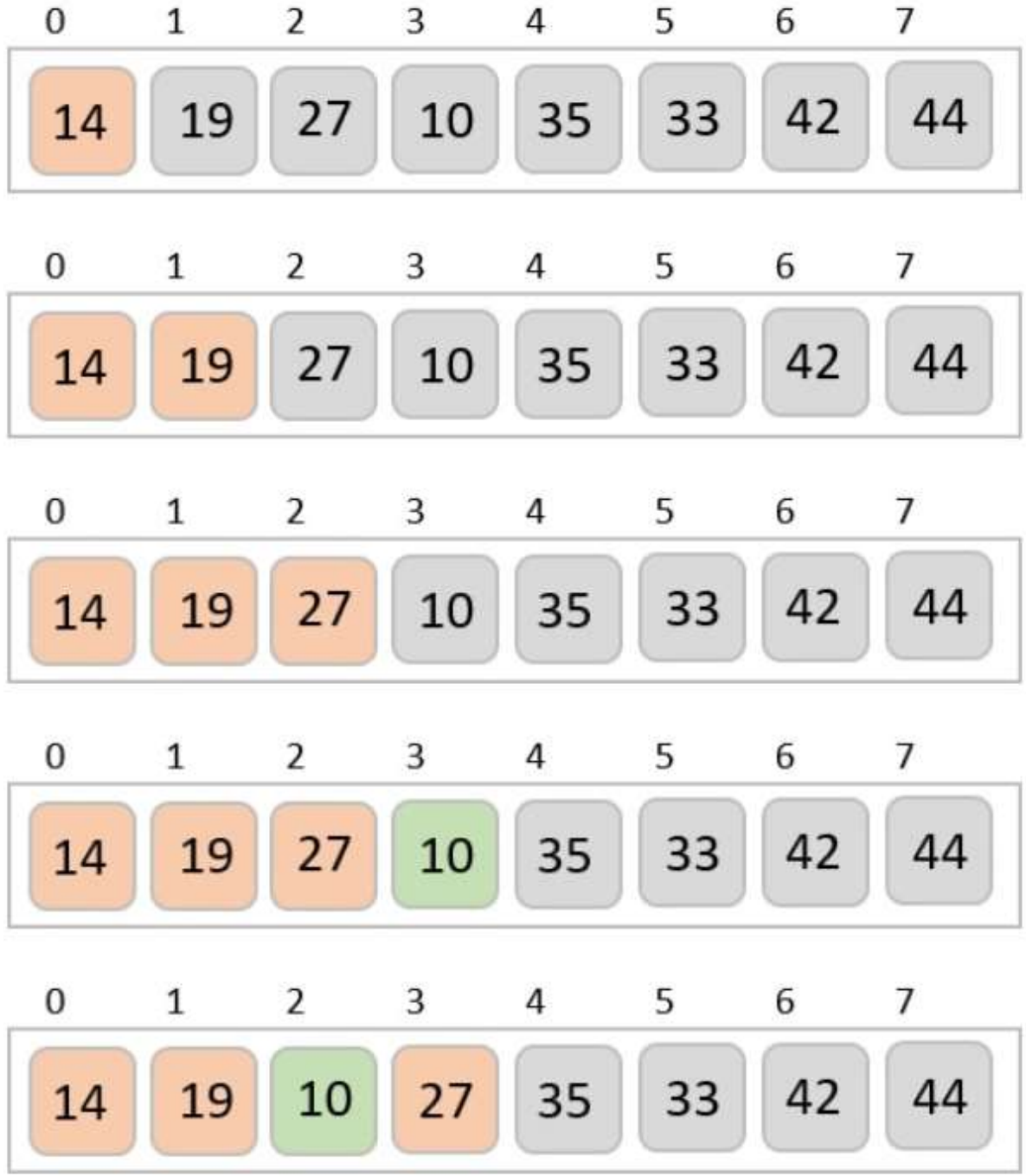
Then, we take interval of 2 and this gap generates two sub-lists - {14, 27, 35, 42}, {19, 10, 33, 44}

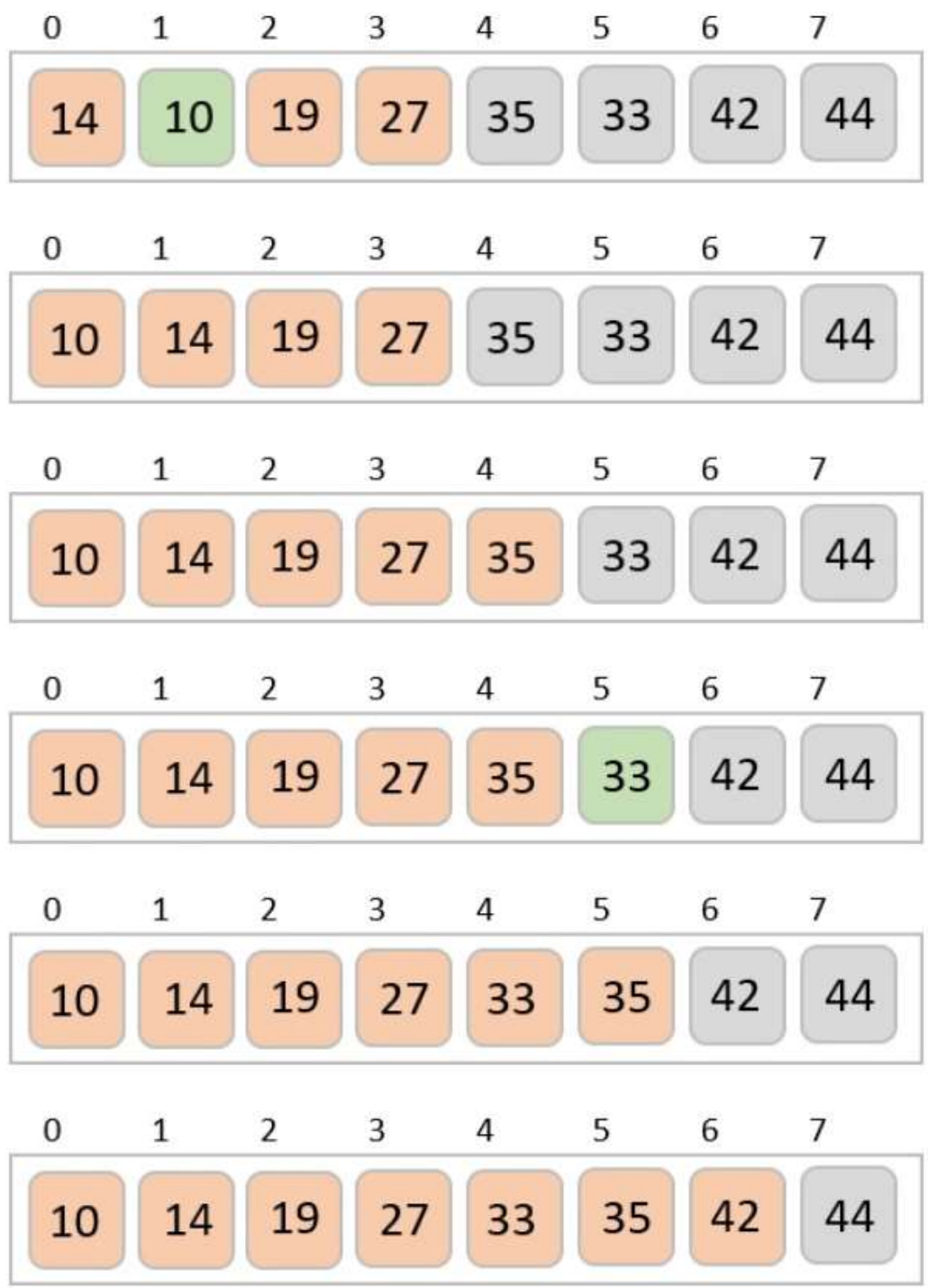


We compare and swap the values, if required, in the original array. After this step, the array should look like this –



Finally, we sort the rest of the array using interval of value 1. Shell sort uses insertion sort to sort the array.


0 1 2 3 4 5 6 7
14 19 27 10 35 33 42 44
0 1 2 3 4 5 6 7
14 19 27 10 35 33 42 44
0 1 2 3 4 5 6 7
14 19 27 10 35 33 42 44
0 1 2 3 4 5 6 7
14 19 27 10 35 33 42 44
0 1 2 3 4 5 6 7
14 19 10 27 35 33 42 44


0 1 2 3 4 5 6 7
14 10 19 27 35 33 42 44
0 1 2 3 4 5 6 7
10 14 19 27 35 33 42 44
0 1 2 3 4 5 6 7
10 14 19 27 35 33 42 44
0 1 2 3 4 5 6 7
10 14 19 27 35 33 42 44
0 1 2 3 4 5 6 7
10 14 19 27 33 35 42 44
0 1 2 3 4 5 6 7
10 14 19 27 33 35 42 44

```cpp
#include<iostream>
using namespace std;
void shellSort(int *arr, int n){
   int gap, j, k;
   for(gap = n/2; gap > 0; gap = gap / 2) { //initially gap = n/2, decreasing by gap /2
      for(j = gap; j<n; j++) {
         for(k = j-gap; k>=0; k -= gap) {
            if(arr[k+gap] >= arr[k])
               break;
            else {
               int temp;
               temp = arr[k+gap];
               arr[k+gap] = arr[k];
               arr[k] = temp;
            }
         }
      }
   }
```

```cpp
int main(){
    int n;
    n = 5;
    int arr[5] = {33, 45, 62, 12, 98}; // initialize the array
    cout << "Array before Sorting: ";
    for(int i = 0; i<n; i++)
        cout << arr[i] << " ";
    cout << endl;
    shellSort(arr, n);
    cout << "Array after Sorting: ";
    for(int i = 0; i<n; i++)
        cout << arr[i] << " ";
    cout << endl;
}
```